

Malware Analysis Report

CS6264 Lab 02

Malware 1: mydoom1.exe

I analyzed `mydoom1.exe` in Ghidra and located `FUN_00804d32` at `0x00804D32`. This function performs file system and time operations. It validates arguments and waits on a flag before lowercasing the directory path and checking for trigger words. It also enumerates files and recursively searches through subdirectories. I followed the matching branch at `0x00804DDE` to the copy handler.

I then used `malware1_inputs.py` with angr to generate inputs that would reach this branch. As a result, the script modeled 20 lowercase characters with recursion depth 1 and a cleared wait flag. It produced inputs covering `shar` and `download` as well as `ftproot` and `incoming`. These results established the words as directory-path triggers without establishing a network C2 role.

For concrete testing I first recorded an instrumented baseline that did not reach the matching block. I then restored a clean Windows snapshot and modified and rebuilt the tracer to inject a path containing `shar`. With the supplied mutex-check bypass enabled the run reached the matching block and created `Winamp 5.0 (en) Crack.exe` and `Kazaa Lite.exe` along with `index.com`. All three `CopyFileA` calls returned 1 and the directory listing confirmed the files were created. I recorded the findings in Excel and retained the trace and directory screenshots as evidence. Concrete testing covered only `shar` while symbolic analysis covered all four triggers.

Malware 2: win33.exe

For the second part of the project I analyzed `win33.exe` in Ghidra and located `FUN_00404ad0` at `0x00404AD0`. I then followed its helper calls to better understand the matching conditions. I saw that the digit helper accepts numbers with 15, 16 or 19 digits. I also found that `FUN_004047e0` applies validation through alternating digit calculations and a final modulo 10 check. Other helpers validated the name field in the `^` branch and the numeric ending. Ultimately, the parser recognizes `^` and `=` as well as `D`. These characters are record separators.

Next, I followed cross-references to `FUN_00404130` and found that it enumerates processes and uses `VirtualQueryEx` and `ReadProcessMemory` before calling the parser this explained where the input buffers originated. I traced the `StrStrA` duplicate check and the two collection calls that followed it. Then I saw that `FUN_00404340` appends new records to the history buffer and `FUN_00404650` appends them to a separate collection buffer that can grow through `VirtualAlloc`. After tracing these calls I finally was able to identify `0x00404340` as a useful target for finding accepted inputs.

I used `malware2_inputs.py` with `angr` to solve a 16 digit ASCII number and a separator while keeping the remaining fields fixed. By using an empty history buffer modeled we were able to get the script to reach `0x00404340` for each of the three separators (as shown in the screen grabs). The initial concrete trace showed `win33.exe` launching Internet Explorer and calling `WriteProcessMemory` and `CreateRemoteThread`. After this I was able to trace the child process and adjust the hooks for the injected image base. Finally I injected the three generated test inputs into the parser. The results were good! I was able to get `WIN33_MATCH` and `WIN33_STORED` confirming that all three inputs were accepted and appended to the internal history buffer.

Malware 3: `unknown.exe1`

For some reason what I was analyzing was `unknown.exe1` instead of `unknown.exe` in Ghidra. I think it should have been `exe` but that was only minor. It still worked. I ended up following its startup code to an export lookup routine. The program calculates numeric values from export names and compares them with stored values after an XOR operation. This explains the indirect function pointers. I followed the control loop to `0x0040140A`. Its request pattern and status check for 200 were consistent with HTTP communication. I then located the response dispatcher `FUN_0040e9f0` at `0x0040E9F0`.

The dispatcher selects three handlers using numeric values. The call arguments indicate that branch 1 writes the supplied data to a file and then attempts to launch it. Branch 2 uses the same writer and a launch helper with a user token path. Branch 3 maps a PE image into executable memory and starts a thread that calls the image entry point. I checked the entry point calculation in `FUN_00401939`. The thread first passes argument 1 and then passes 10 with `DAT_00413cd4` after a nonzero return. The indirect API names were inferred from their arguments and Windows structure layouts.

I followed the record parser `FUN_0040cf97`. Its second field supplies the branch selector and its third field supplies the data pointer and length. These are numeric selectors within structured records. The exact byte encoding has not been verified as I ran out of time for this project (this project took a lot of time to handle each part). I prepared `malware3_inputs.py` to search for inputs reaching the three handler entries with `angr`. But I was just getting errors. So instead I am just going to upload the images and record the static findings in Excel.